

Classification: K-Nearest Neighbor, Naïve Bayes, Decision Tree, Random Forest, Support Vector Machine

Case study on advanced supervised ML techniques for regression and classification

Decision Tree algorithm

- Decision trees are a simple machine learning tool used for **classification and regression tasks**.
- They break complex decisions into smaller steps, making them easy to understand and implement.

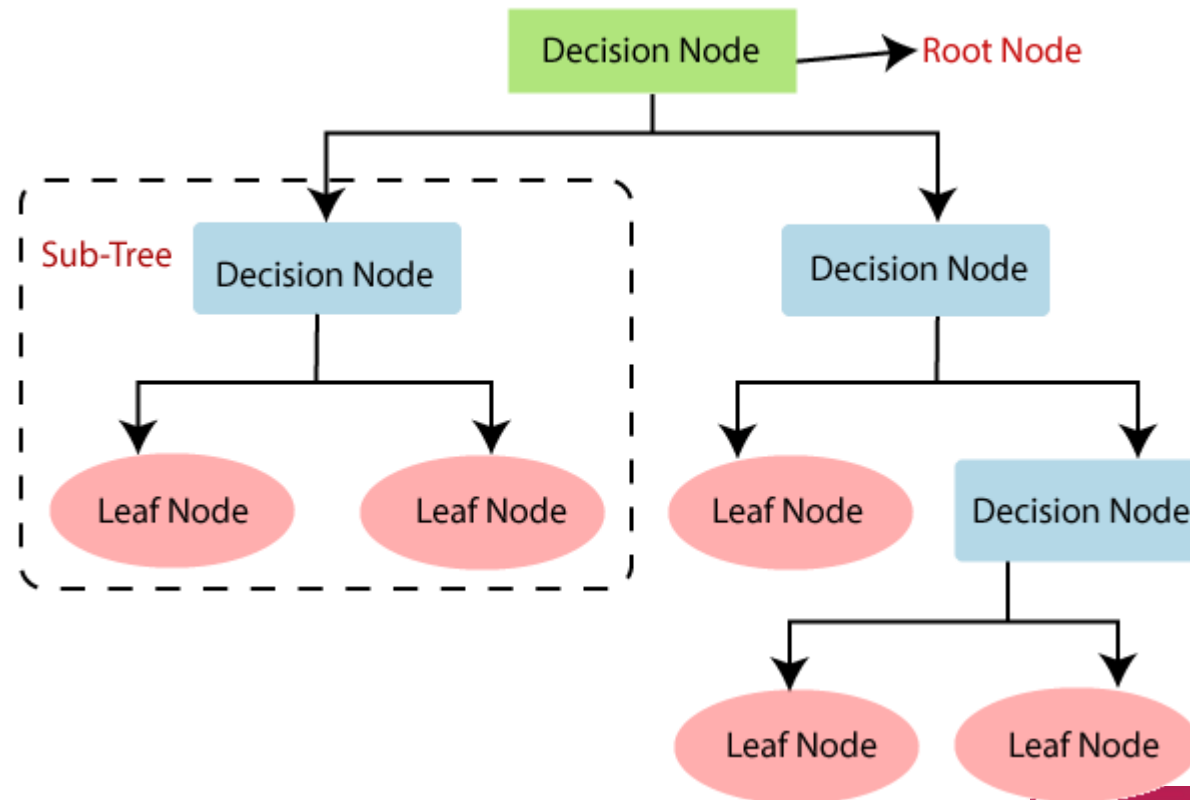


Decision Tree

- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**
- It is a tool that has applications spanning several different areas. These trees can be used for classification as well as regression problems.
- The name itself suggests that it uses a flowchart like a tree structure to show the predictions that result from a series of feature-based splits. It starts with a root node and ends with a decision made by leaves.
- two nodes-**Decision Node** and **Leaf Node**.
- Decision nodes used to make any decision and have multiple branches.
- Leaf nodes are the output of those decisions and do not contain any further branches.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***



- In order to build a tree, use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.



- Why use Decision Trees?

Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.

The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

Root Node: Root node is from where the decision tree starts. It represents the entire dataset

Leaf Node: Leaf nodes are the final output node.

Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

Branch/Sub Tree: A tree formed by splitting the tree.

Pruning: Pruning is the process of removing the unwanted branches from the tree.

Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.



How does the Decision Tree algorithm Work?

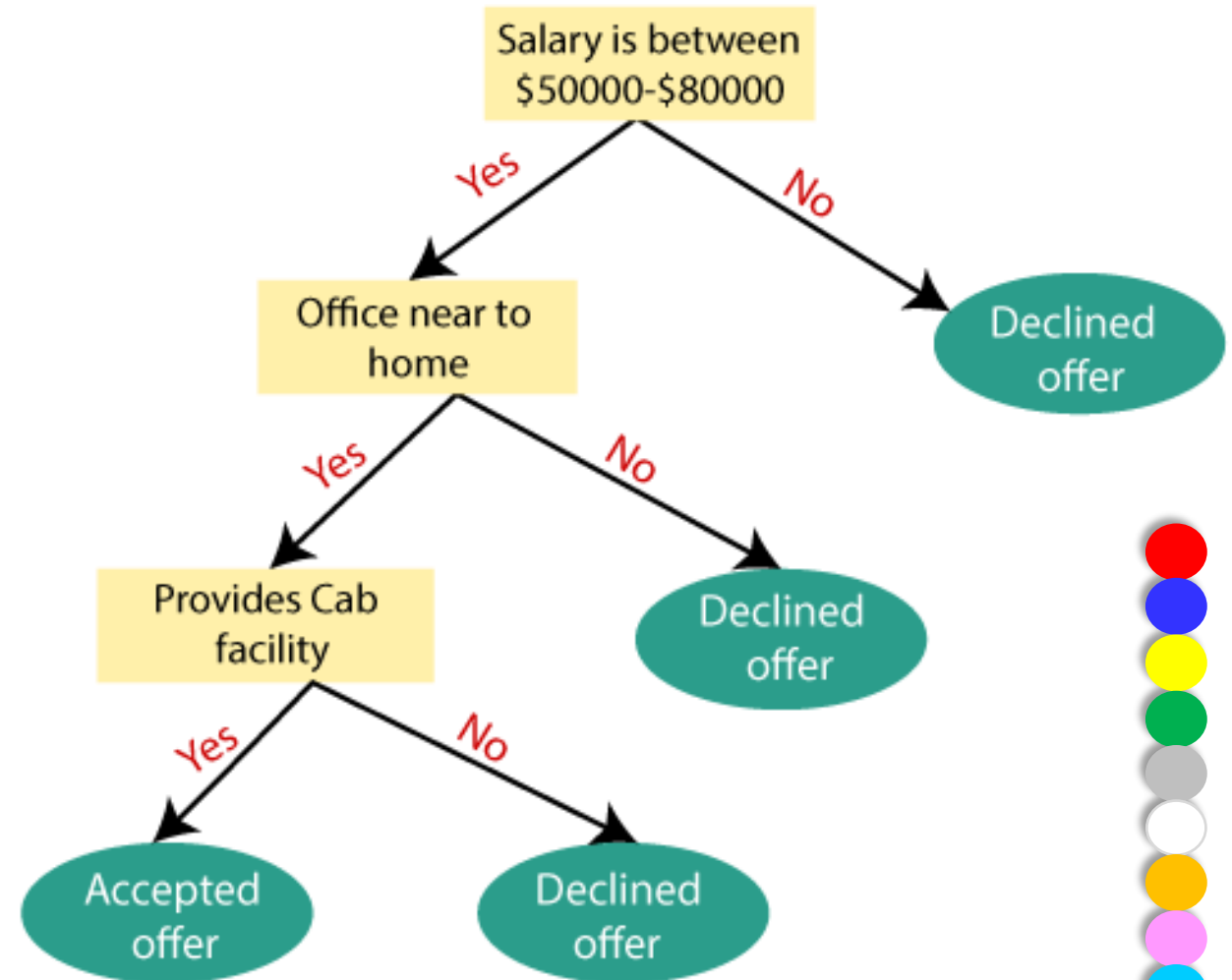
Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using **Attribute Selection Measure (ASM) (CART)**.

Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.



- basic algorithm ID3 learns decision trees by constructing them top-down, beginning with the question, “**Which attribute should be tested at the root of the tree?**”
- To answer this question, each attribute is evaluated using a **statistical test** to determine how well it alone classifies the training examples.
- The **best attribute is selected as the root of the tree.**
- Our next task is to find which node will be next after root. In this case, we would like to again choose the attribute which is most useful to classify training examples.
- Then repeat the process until we find **leaf node.**



Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



How to know it is a pure split?

- Entropy
- Gini impurity



Entropy

- Entropy is the measurement of **homogeneity**. It returns us the information about an arbitrary dataset that how impure/non-homogeneous the data set is.
- Given a collection of examples/dataset S, containing positive and negative examples of some target concept, the entropy of S relative to classification is

$$Entropy(S) = -(P_{\oplus} \log_2 P_{\oplus} + P_{\ominus} \log_2 P_{\ominus}) \quad (1.1)$$

Where $P_{\oplus}$ is the portion of positive examples and $P_{\ominus}$ is the portion of negative examples in S.



- To illustrate this equation, we will do an example that calculates the entropy of our data set in Fig: 1. The dataset has 9 positive instances and 5 negative instances, therefore-

$$Entropy([9+, 5-]) = -(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14}) = 0.940 \quad (1.2)$$

Which concludes, the dataset is 94% impure or 94% non-homogeneous.

Let's do some more calculations and try to understand the nature of *Entropy*. What could be the Entropy of [7+, 7-] & [14+, 0-]?

$$Entropy[7+, 7-] = -(\frac{7}{14} \log_2 \frac{7}{14} + \frac{7}{14} \log_2 \frac{7}{14}) = 1 \quad (1.3)$$

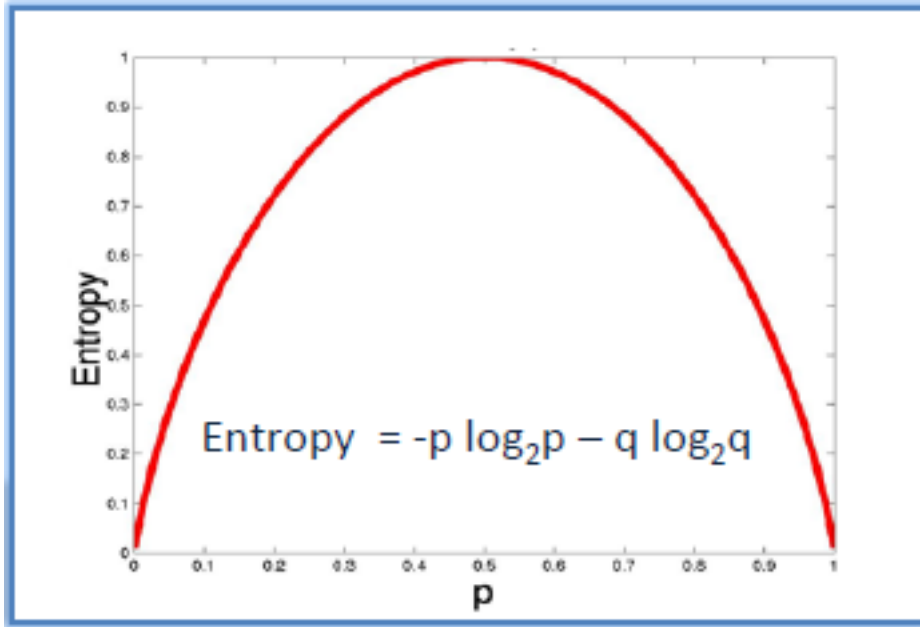
And,

$$Entropy[14+, 0-] = -(\frac{14}{14} \log_2 \frac{14}{14} + \frac{0}{14} \log_2 \frac{0}{14}) = 0 \quad (1.4)$$

By observing closely on equations 1.2, 1.3 and 1.4; we can come to a conclusion that if the data set is **completely homogeneous (pure split) then the impurity is 0, therefore entropy is 0** (equation 1.4), but if the data set can be equally divided into two classes, then it is completely non-homogeneous & impurity is 100%, therefore entropy is 1 (equation 1.3).



Entropy



Case 1: When $p = 0$ or $p = 1$

- One event is certain.
- No uncertainty.
- Entropy = 0

Example:

- If a coin always gives heads (probability = 1)

Case 2: When $p = 0.5$

- Both outcomes equally likely.
- Maximum uncertainty.
- Entropy = 1 bit (maximum for binary case).



Information Gain:

- Given Entropy is the measure of impurity in a collection of a dataset, now we can measure the effectiveness of an attribute in classifying the training set.
- The measure we will use called **information gain**, is simply the expected reduction in **entropy** caused by partitioning the data set according to this attribute.
- The information gain (**$Gain(S,A)$**) of an attribute A relative to a collection of data set S , is defined as-

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \quad (1.5)$$

Where, $Values(A)$ is the all possible values for attribute A , and S_v is the subset of S for which attribute A has value v .



Gini impurity

- Gini impurity measures how mixed the classes are in a node and is used in CART to choose the best split.

$$Gini = 1 - \sum p_i^2$$

Suppose:

10 Yes

0 No

$$p_{yes} = 1, \quad p_{no} = 0$$

$$Gini = 1 - (1^2 + 0^2)$$

$$Gini = 0$$

Gini = 0 ; Perfectly Pure

Suppose:

•5 Yes

•5 No

$$p_{yes} = 0.5, \quad p_{no} = 0.5$$

$$Gini = 1 - (0.5^2 + 0.5^2)$$

$$Gini = 1 - (0.25 + 0.25)$$

$$Gini = 0.5$$

Maximum impurity (binary case)



```

# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
    #importing datasets
data_set= pd.read_csv('user_data.csv')
    #Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
    # Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test= train_test_split(x, y, test_size= 0.2
5,random_state=0)
    #feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)

```



Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0
15	15697686	Male	29	80000	0

Fitting a Decision-Tree algorithm to the Trainingset

#Fitting Decision Tree classifier to the training set

From sklearn.tree **import** DecisionTreeClassifier

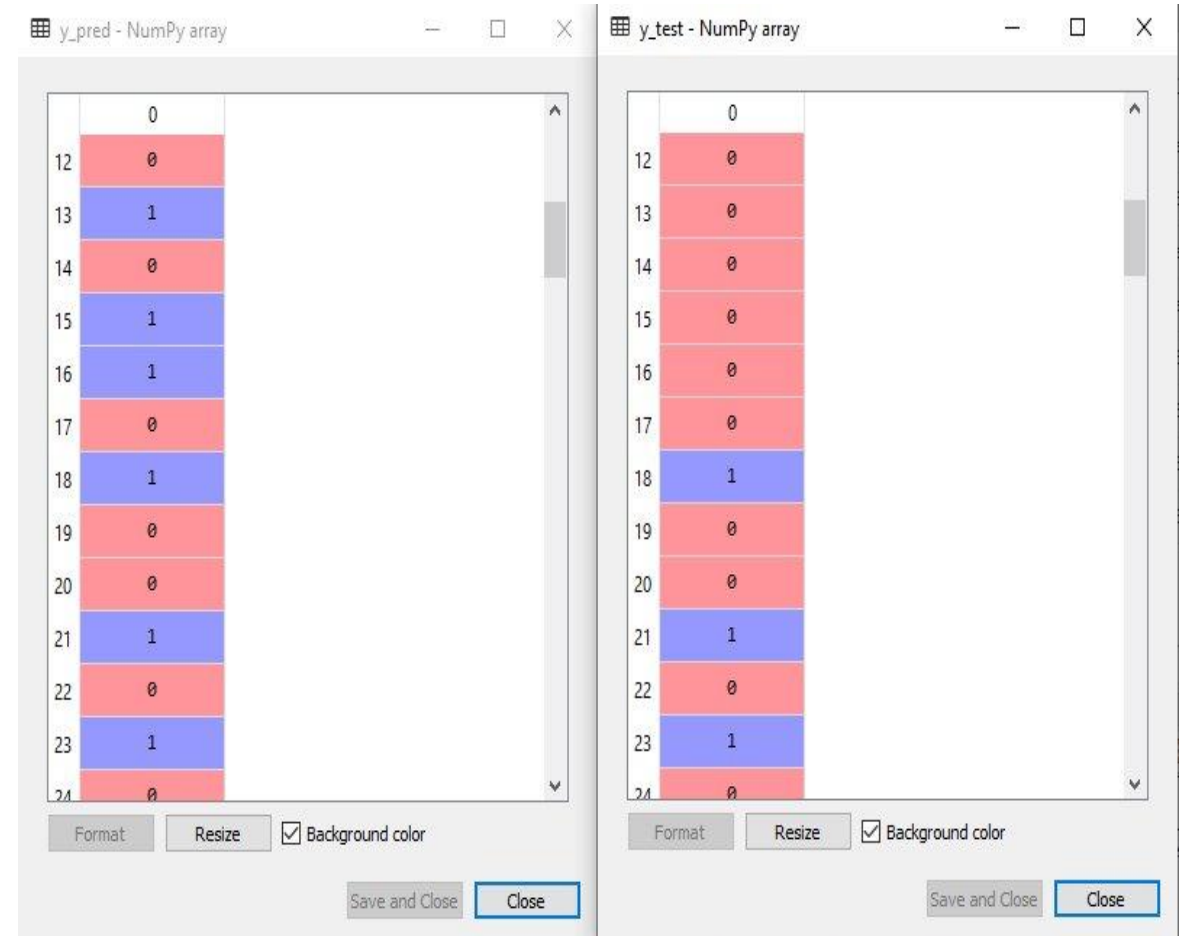
```
classifier= DecisionTreeClassifier(criterion='entropy',  
random_state=0)
```

```
classifier.fit(x_train,y_train)
```

Predicting the test result

#Predicting the test set result

```
y_pred= classifier.predict(x_test)
```

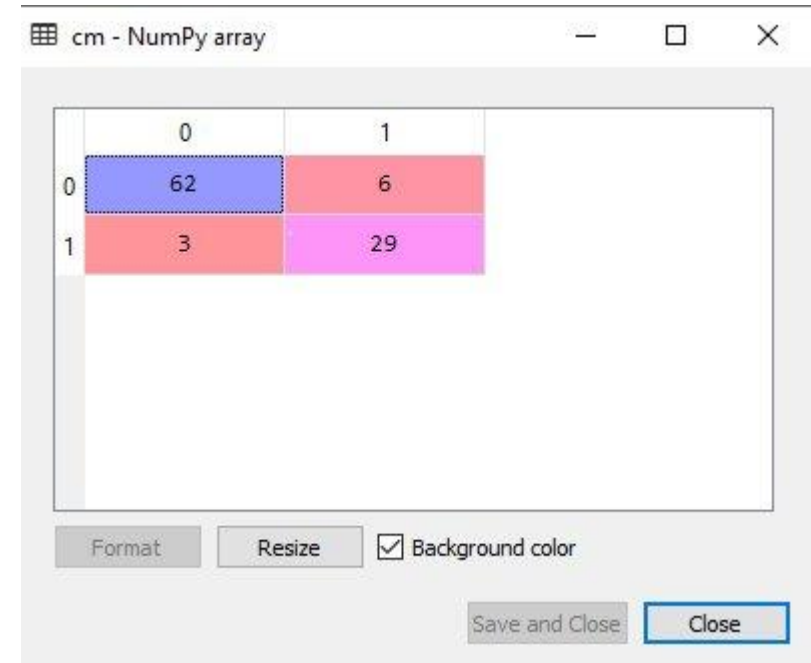


Test accuracy of the result

#Creating the Confusion matrix

from sklearn.metrics **import** confusion_matrix

cm= confusion_matrix(y_test,y_pred)



CART & ID3 Algorithm

- *CART (Classification and Regression Trees) and ID3 (Iterative Dichotomiser 3) are two fundamental algorithms for creating decision trees in machine learning. They are used to split datasets into subsets based on attribute values, eventually forming a tree structure that can be used for classification or regression.*



ID3 Algorithm-Iterative Dichotomiser 3

Key Characteristics

- **Uses Entropy and Information Gain:** ID3 uses entropy as a measure of impurity and information gain to decide which attribute to split the data on at each step.
- **Categorical Data:** ID3 is designed to work with categorical data. Continuous data needs to be discretized before using ID3.
- **Greedy Approach:** The algorithm uses a top-down, recursive approach, selecting the best attribute based on the highest information gain at each step.

Steps of ID3 Algorithm

- **Calculate Entropy:** Calculate the entropy of the target class in the dataset.
- **Calculate Information Gain:** For each feature, calculate the information gain
- **Select Best Attribute:** Choose the attribute with the highest information gain to split the dataset.
- **Create Subtrees:** Split the dataset into subsets using the selected attribute and repeat the process recursively for each subset.
- **Stopping Criteria:** The recursion stops when all data points belong to a single class, or there are no more attributes to split on.



CART Algorithm

- **CART** stands for Classification and Regression Trees and is a more advanced algorithm that can handle both classification and regression tasks
- **Key Characteristics**
- **Uses Gini Impurity or MSE:** *For classification tasks, CART uses the Gini impurity measure to decide splits, while for regression tasks, it uses mean squared error (MSE).*
- **Handles Numerical and Categorical Data:** *Unlike ID3, CART can handle both categorical and continuous data without requiring prior discretization.*
- **Binary Trees:** *CART produces binary trees, meaning each internal node splits the data into two child nodes.*



Steps of CART Algorithm:

- **Select the Best Split:** For each feature, evaluate all possible splits and select the one that results in the largest reduction in impurity (Gini impurity for classification, MSE for regression).
- **Split the Dataset:** Use the selected split to partition the dataset into two subsets.
- **Create Subtrees:** Recursively apply the same procedure to the subsets to grow the tree.
- **Pruning:** After the tree is fully grown, apply pruning to remove nodes that do not provide additional information, helping to prevent overfitting.
- **Stopping Criteria:** The tree growing stops when further splits do not significantly improve the model, or a specified depth is reached.



Differences Between ID3 and CART

- **Data Types:** ID3 primarily works with categorical data, while CART can handle both categorical and numerical data.
- **Splitting Criterion:** ID3 uses information gain based on entropy, whereas CART uses Gini impurity for classification and MSE for regression.
- **Tree Structure:** ID3 can create multi-way splits, whereas CART produces binary trees.
- **Pruning:** CART has built-in pruning methods to prevent overfitting, while ID3 does not inherently include pruning, though it can be applied manually.
- **Use Cases**
 - **ID3:** Suitable for simple classification tasks with categorical data.
 - **CART:** Versatile for both classification and regression tasks, handling complex datasets with mixed data types.



Problem

Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No



Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Outlook

Values (Outlook) = Sunny, Overcast, Rain

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Sunny} \leftarrow [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

$$S_{Overcast} \leftarrow [4+, 0-]$$

$$Entropy(S_{Overcast}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

$$S_{Rain} \leftarrow [3+, 2-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$Gain(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook)$$

$$= Entropy(S) - \frac{5}{14} Entropy(S_{Sunny}) - \frac{4}{14} Entropy(S_{Overcast})$$

$$- \frac{5}{14} Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14} 0.971 - \frac{4}{14} 0 - \frac{5}{14} 0.971 = 0.2464$$



Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Hot} \leftarrow [2+, 2-]$$

$$Entropy(S_{Hot}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.0$$

$$S_{Mild} \leftarrow [4+, 2-]$$

$$Entropy(S_{Mild}) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

$$S_{Cool} \leftarrow [3+, 1-]$$

$$Entropy(S_{Cool}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$Gain(S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp)$$

$$= Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild}) - \frac{4}{14} Entropy(S_{Cool})$$

$$Gain(S, Temp) = 0.0289$$



Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{High} \leftarrow [3+, 4-]$$

$$Entropy(S_{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$$

$$S_{Normal} \leftarrow [6+, 1-]$$

$$Entropy(S_{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5916$$

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity)$$

$$= Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14} 0.9852 - \frac{7}{14} 0.5916$$

$$Gain(S, Humidity) = 0.1516$$



Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Wind

Values (Wind) = Strong, Weak

$$S = [9+, 5-] \quad Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Strong} \leftarrow [3+, 3-] \quad Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [6+, 2-] \quad Entropy(S_{Weak}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

$$Gain(S, Wind) = 0.94 - \frac{6}{14} 1.0 - \frac{8}{14} 0.8113$$

$$Gain(S, Wind) = 0.0478$$



Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

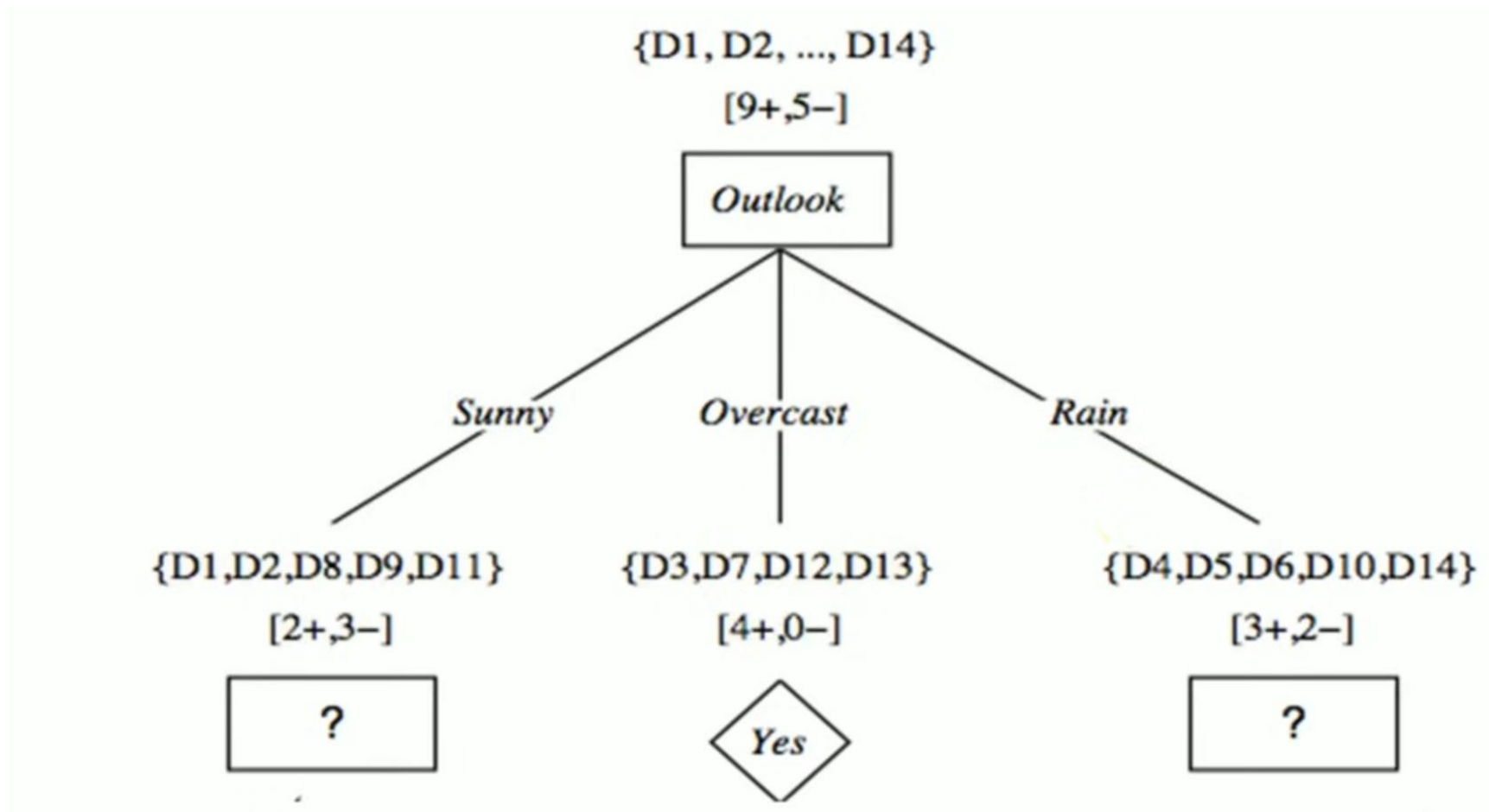
$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$





Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 2-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [1+, 1-]$$

$$Entropy(S_{Mild}) = 1.0$$

$$S_{Cool} \leftarrow [1+, 0-]$$

$$Entropy(S_{Cool}) = 0.0$$

$$Gain(S_{Sunny}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp)$$

$$= Entropy(S) - \frac{2}{5} Entropy(S_{Hot}) - \frac{2}{5} Entropy(S_{Mild})$$

$$- \frac{1}{5} Entropy(S_{Cool})$$

$$Gain(S_{Sunny}, Temp) = 0.97 - \frac{2}{5} 0.0 - \frac{2}{5} 1 - \frac{1}{5} 0.0$$

$$Gain(S_{Sunny}, Temp) = 0.570$$



Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{high} \leftarrow [0+, 3-]$$

$$Entropy(S_{High}) = 0.0$$

$$S_{Normal} \leftarrow [2+, 0-]$$

$$Entropy(S_{Normal}) = 0.0$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \frac{3}{5} Entropy(S_{High}) - \frac{2}{5} Entropy(S_{Normal})$$

$$Gain(S_{Sunny}, Humidity) = 0.97 - \frac{3}{5} 0.0 - \frac{2}{5} 0.0$$

$$Gain(S_{Sunny}, Humidity) = 0.97$$



Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Wind

Values (Wind) = Strong, Weak

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{Strong} \leftarrow [1+, 1-]$$

$$Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [1+, 2-]$$

$$Entropy(S_{Weak}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.9183$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

$$Gain(S_{Sunny}, Wind) = 0.97 - \frac{2}{5} 1.0 - \frac{3}{5} 0.918 = 0.0192$$



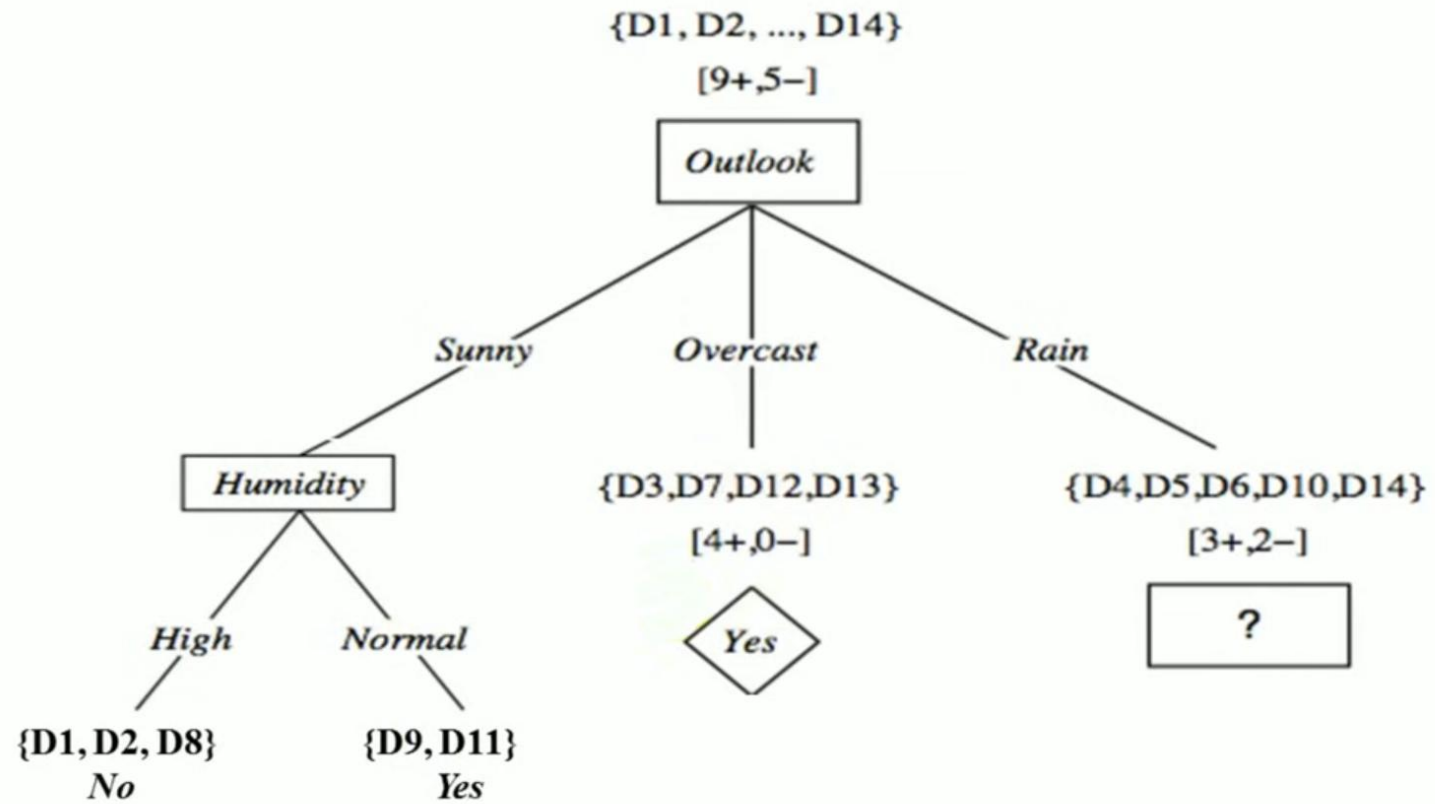
Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$





Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 0-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [2+, 1-]$$

$$Entropy(S_{Mild}) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.9183$$

$$S_{Cool} \leftarrow [1+, 1-]$$

$$Entropy(S_{Cool}) = 1.0$$

$$Gain(S_{Rain}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp)$$

$$= Entropy(S) - \frac{0}{5} Entropy(S_{Hot}) - \frac{3}{5} Entropy(S_{Mild})$$

$$- \frac{2}{5} Entropy(S_{Cool})$$

$$Gain(S_{Rain}, Temp) = 0.97 - \frac{0}{5} 0.0 - \frac{3}{5} 0.918 - \frac{2}{5} 1.0$$

$$Gain(S_{rain}, Temp) = 0.0192$$



Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$$

$$S_{High} \leftarrow [1+, 1-]$$

$$Entropy(S_{High}) = 1.0$$

$$S_{Normal} \leftarrow [2+, 1-]$$

$$Entropy(S_{Normal}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \frac{2}{5} Entropy(S_{High}) - \frac{3}{5} Entropy(S_{Normal})$$

$$Gain(S_{Rain}, Humidity) = 0.97 - \frac{2}{5} 1.0 - \frac{3}{5} 0.918$$

$$Gain(S_{rain}, Humidity) = 0.0192$$



Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Wind

Values (wind) = Strong, Weak

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2 \frac{3}{5} - \frac{2}{5}\log_2 \frac{2}{5} = 0.97$$

$$S_{Strong} \leftarrow [0+, 2-]$$

$$Entropy(S_{Strong}) = 0.0$$

$$S_{Weak} \leftarrow [3+, 0-]$$

$$Entropy(S_{Weak}) = 0.0$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

$$Gain(S_{Rain}, Wind) = 0.97 - \frac{2}{5} 0.0 - \frac{3}{5} 0.0$$

$$Gain(S_{rain}, Wind) = 0.97$$



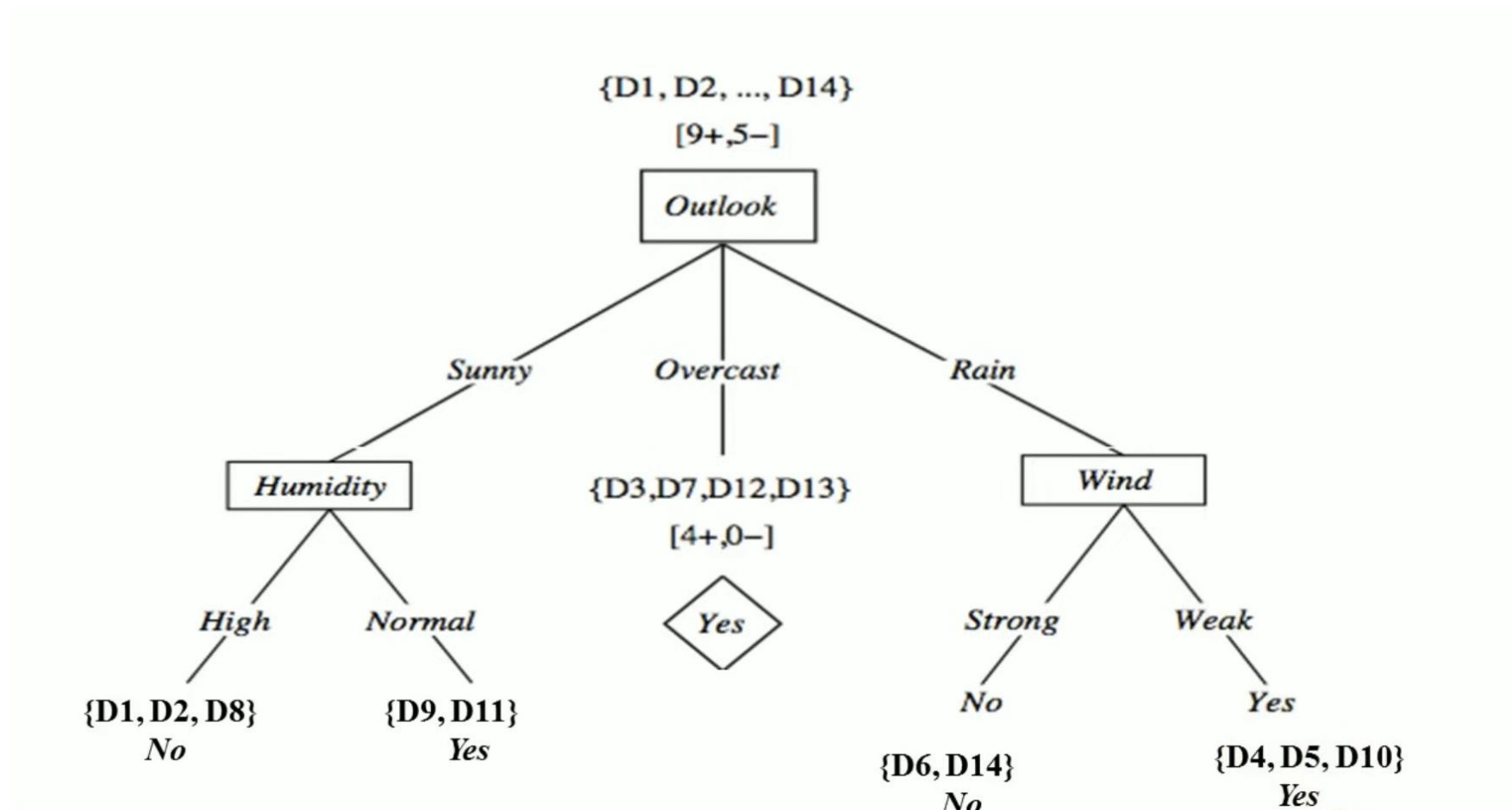
Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

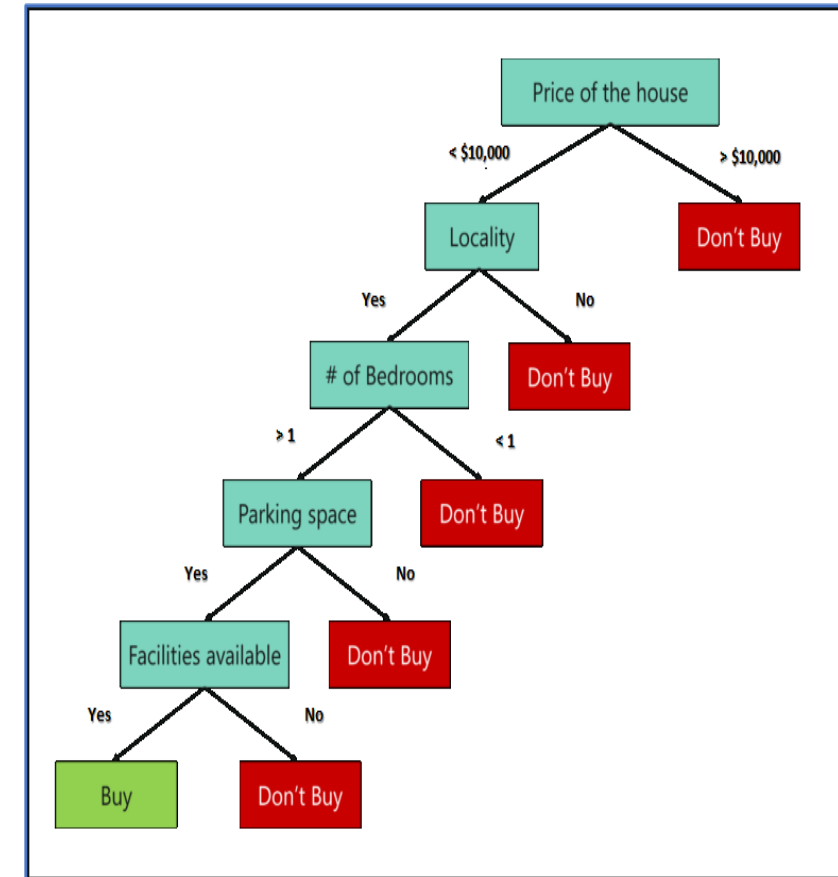
$$(S_{Rain}, Wind) = 0.97$$





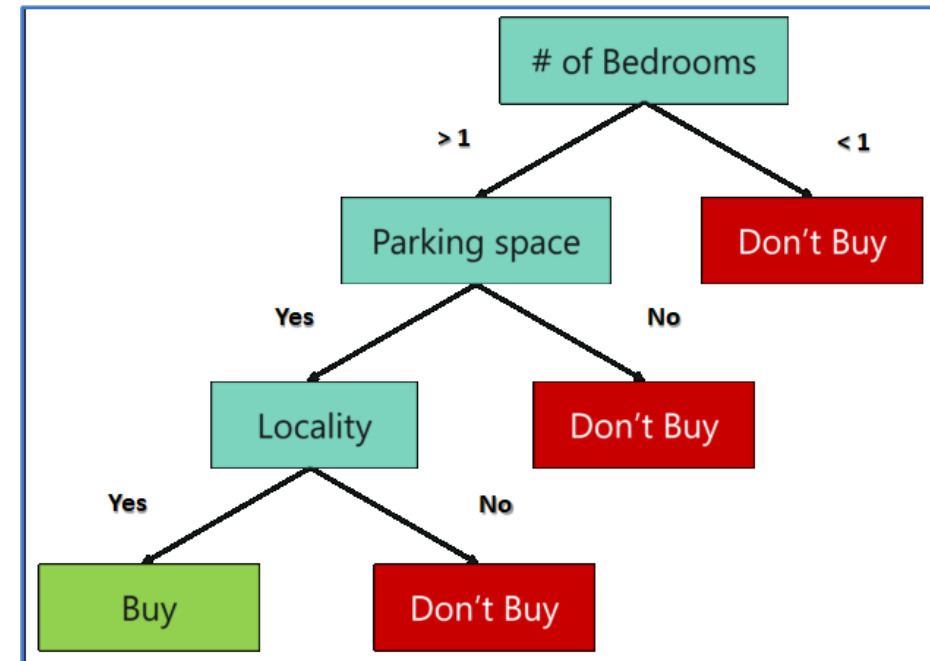
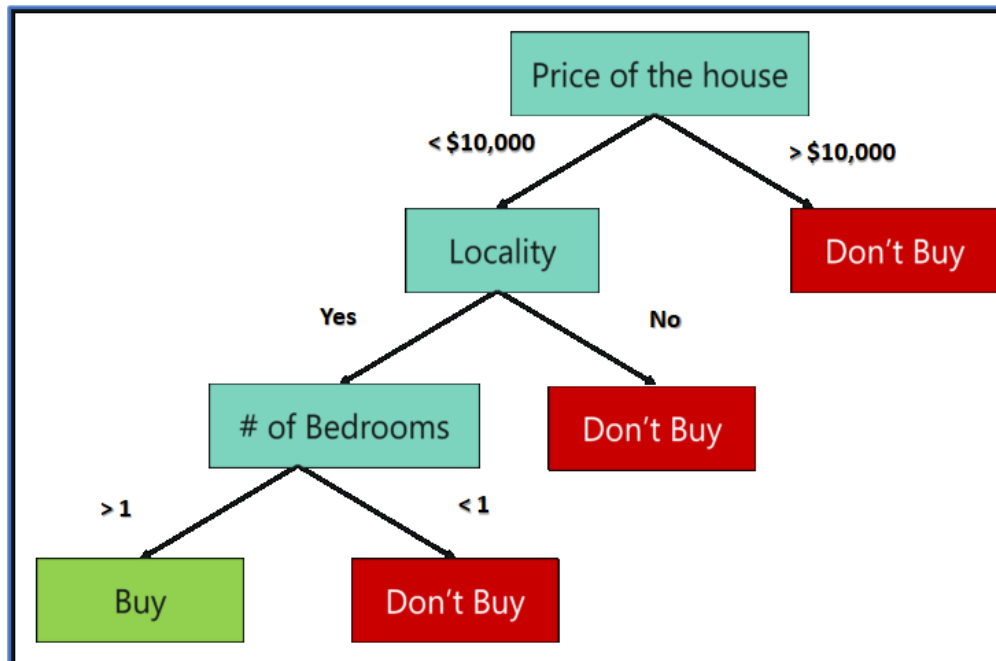
Why use a Random Forest algorithm?

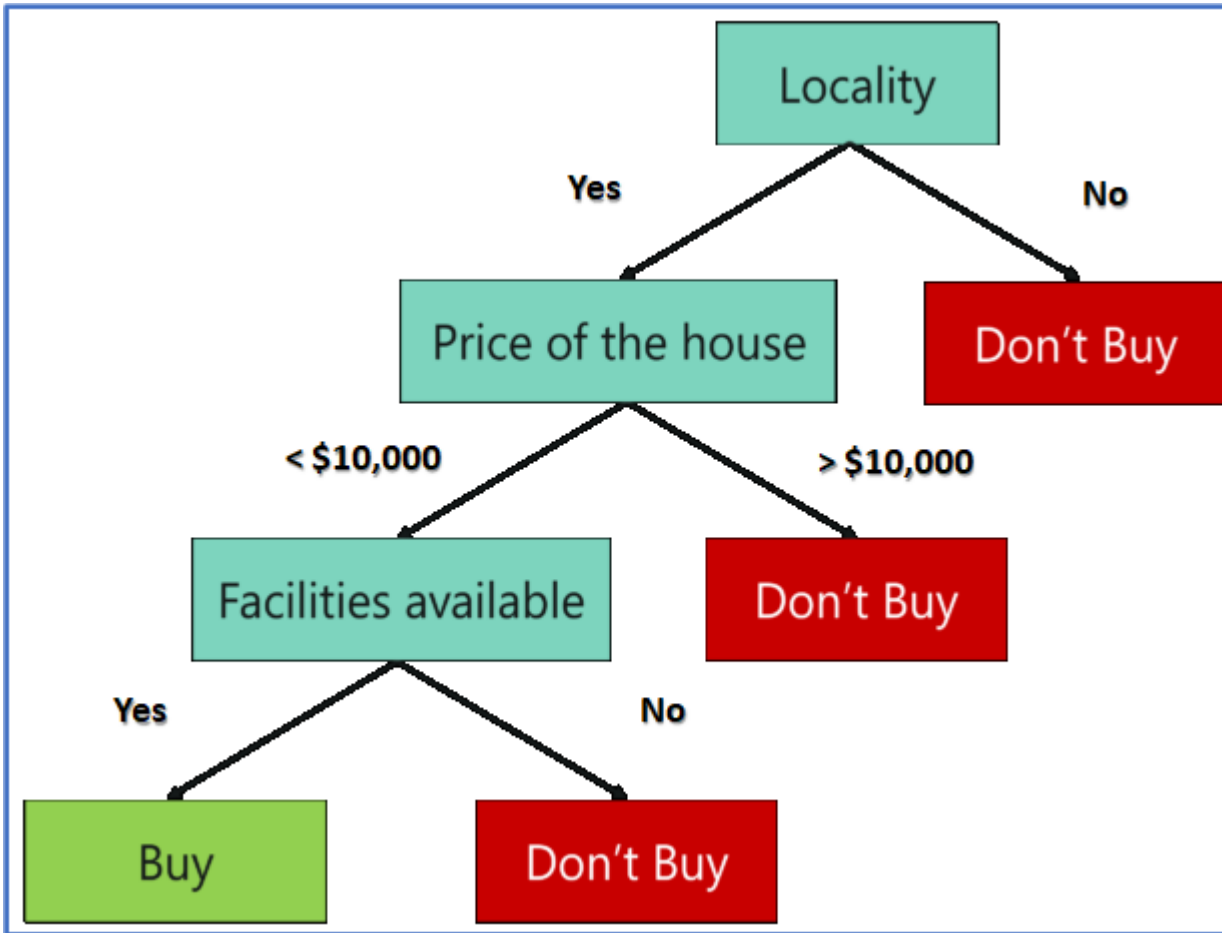
- Even though Decision trees are convenient and easily implemented, they lack accuracy. Decision trees work very effectively with the training data used to build them, but they're not flexible when classifying the new sample. It means that the accuracy during the testing phase is very low. It happens due to a process called Over-fitting.
- ***“Over-fitting occurs when a model studies the training data to such an extent that it negatively influences the model’s performance on new data.”***
- It means that the disturbance in the training data is recorded and learned as concepts by the model. But the problem here is that these concepts do not apply to the testing data and negatively impact the model’s ability to classify the new data, hence reducing the accuracy of the testing data.
- It is where Random Forest comes in. It is based on the idea of bagging, which is used to reduce the variation in the predictions by combining the result of multiple Decision trees on different samples of the data set.



Random Forest

- a Random Forest is an ensemble of decision trees. It randomly selects a set of parameters and creates a decision tree for each set of chosen parameters.





each Decision Tree is taking only three parameters from the entire data set. Each decision tree predicts the outcome based on the respective predictor variables used in that tree and finally takes the average of the results from all the decision trees in the random forest.

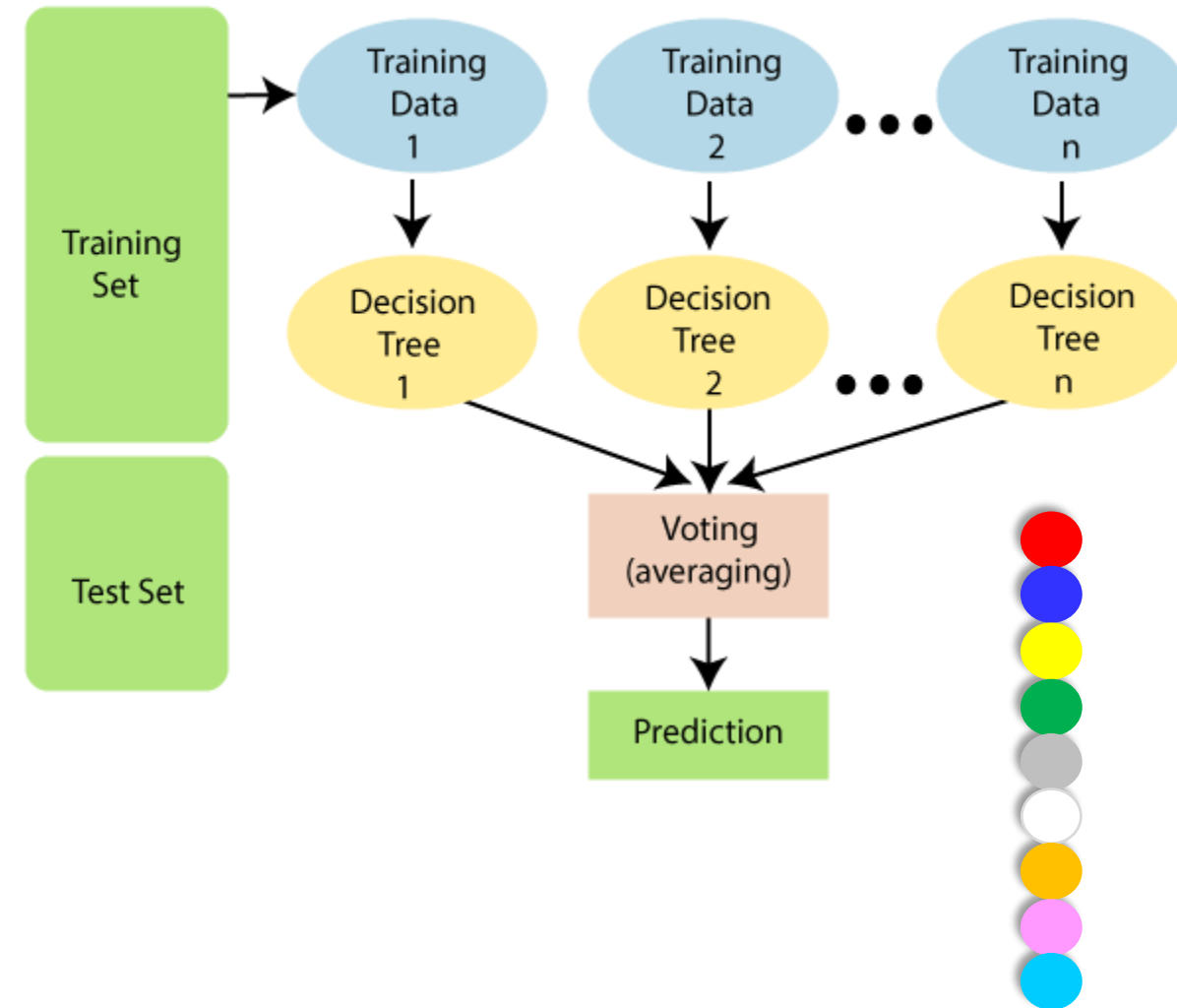


- In simple words, after creating multiple Decision trees using this method, each tree selects or votes the class (in this case, the decision trees will choose whether or not a house is bought). And the class receiving the most votes by a simple majority is termed as the predicted class.
- Decision trees are built on the entire data set using all the predictor variables. In contrast, Random Forests are used to create multiple decision trees, such that each decision tree is built only on the part of the data set.

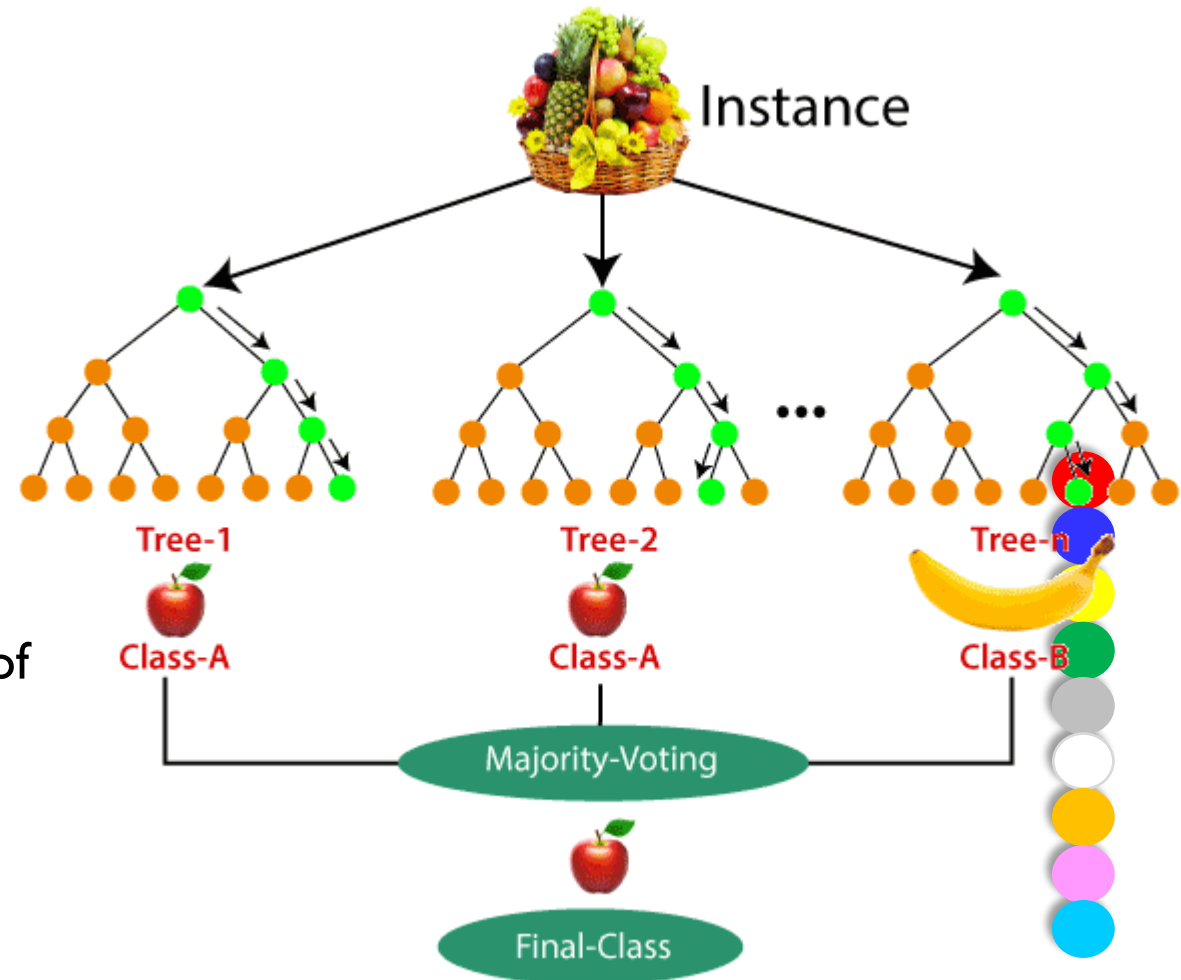


Random Forest

- It is based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem*.
- **Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.**
- The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.



- How does Random Forest algorithm work?
- **Step-1:** Select random K data points from the training set.
- **Step-2:** Build the decision trees associated with the selected data points (Subsets).
- **Step-3:** Choose the number N for decision trees that you want to build.
- **Step-4:** Repeat Step 1 & 2.
- **Step-5:** For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.



Data Pre-Processing Step:

importing libraries

import numpy as nm

import matplotlib.pyplot as mtp

import pandas as pd

#importing datasets

data_set= pd.read_csv('user_data.csv')

#Extracting Independent and dependent Variable

x= data_set.iloc[:, [2,3]].values

y= data_set.iloc[:, 4].values

Splitting the dataset into training and test set.

from sklearn.model_selection **import** train_test_split

x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)

#feature Scaling

from sklearn.preprocessing **import** StandardScaler

st_x= StandardScaler()

x_train= st_x.fit_transform(x_train)

x_test= st_x.transform(x_test)

Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0



Fitting the Random Forest algorithm to the training set:

#Fitting Decision Tree classifier to the training set

from sklearn.ensemble **import** RandomForestClassifier

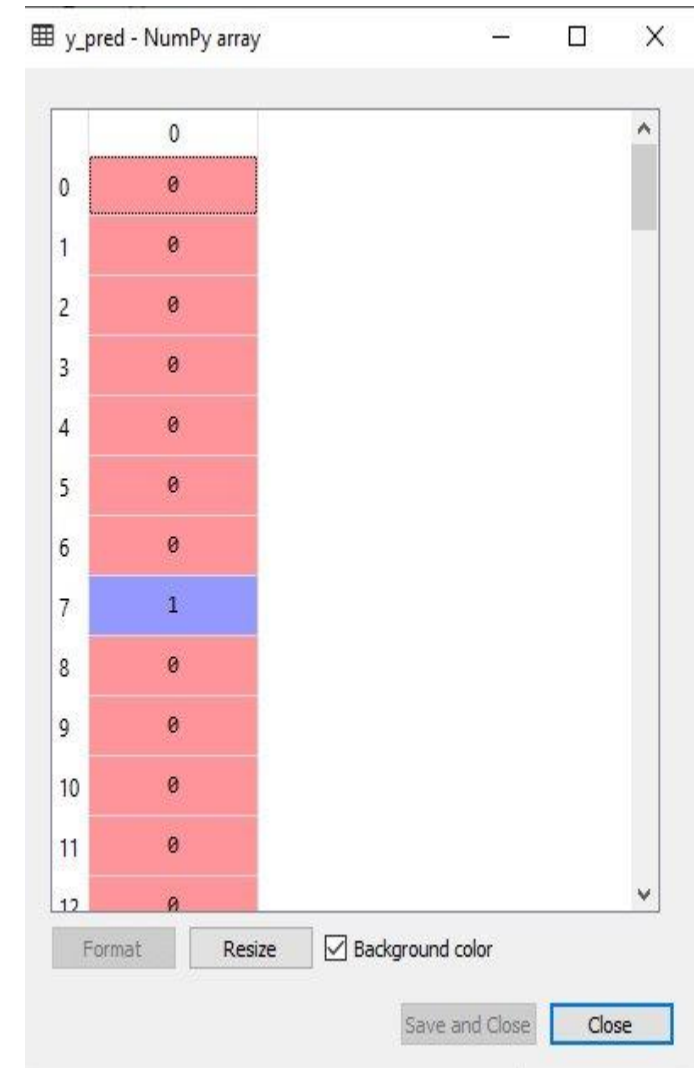
classifier= RandomForestClassifier(n_estimators= 10, criterion="entropy")

classifier.fit(x_train,y_train)

Predicting the Test Set result

#Predicting the test set result

y_pred= classifier.predict(x_test)

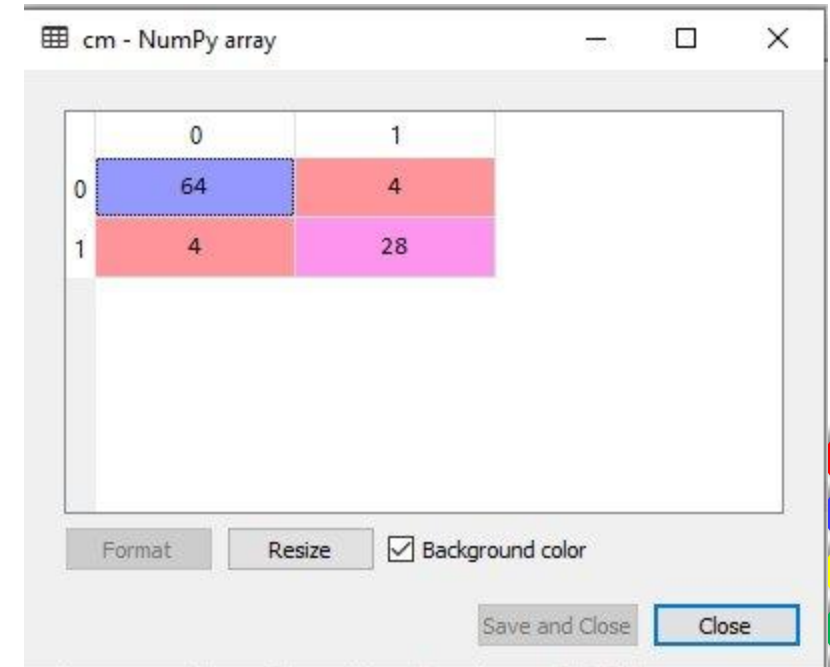


Creating the Confusion Matrix

#Creating the Confusion matrix

from sklearn.metrics **import** confusion_matrix

cm= confusion_matrix(y_test,y_pred)



Random Forest Applications

- **Customer churn prediction:** Businesses can use random forests to predict which customers are likely to churn (cancel their service) so that they can take steps to retain them. For example, a telecom company might use a random forest model to identify customers who are using their phone less frequently or who have a history of late payments.
- **Fraud detection:** Random forests can identify fraudulent transactions in real-time. For instance, a bank might employ a random forest model to spot transactions made from unusual locations or involving unusually large amounts of money.
- **Stock price prediction:** It can predict future [stock prices](#). However, it is important to note that stock price prediction is a very difficult task, and no model is ever going to be perfectly accurate.
- **Medical diagnosis:** These can help doctors diagnose diseases. For example, a doctor might use a random forest model to help them diagnose a patient with cancer.
- **Image recognition:** It can recognize objects in images. For example, a self-driving car might use a random forest model to identify pedestrians and other vehicles on the road.



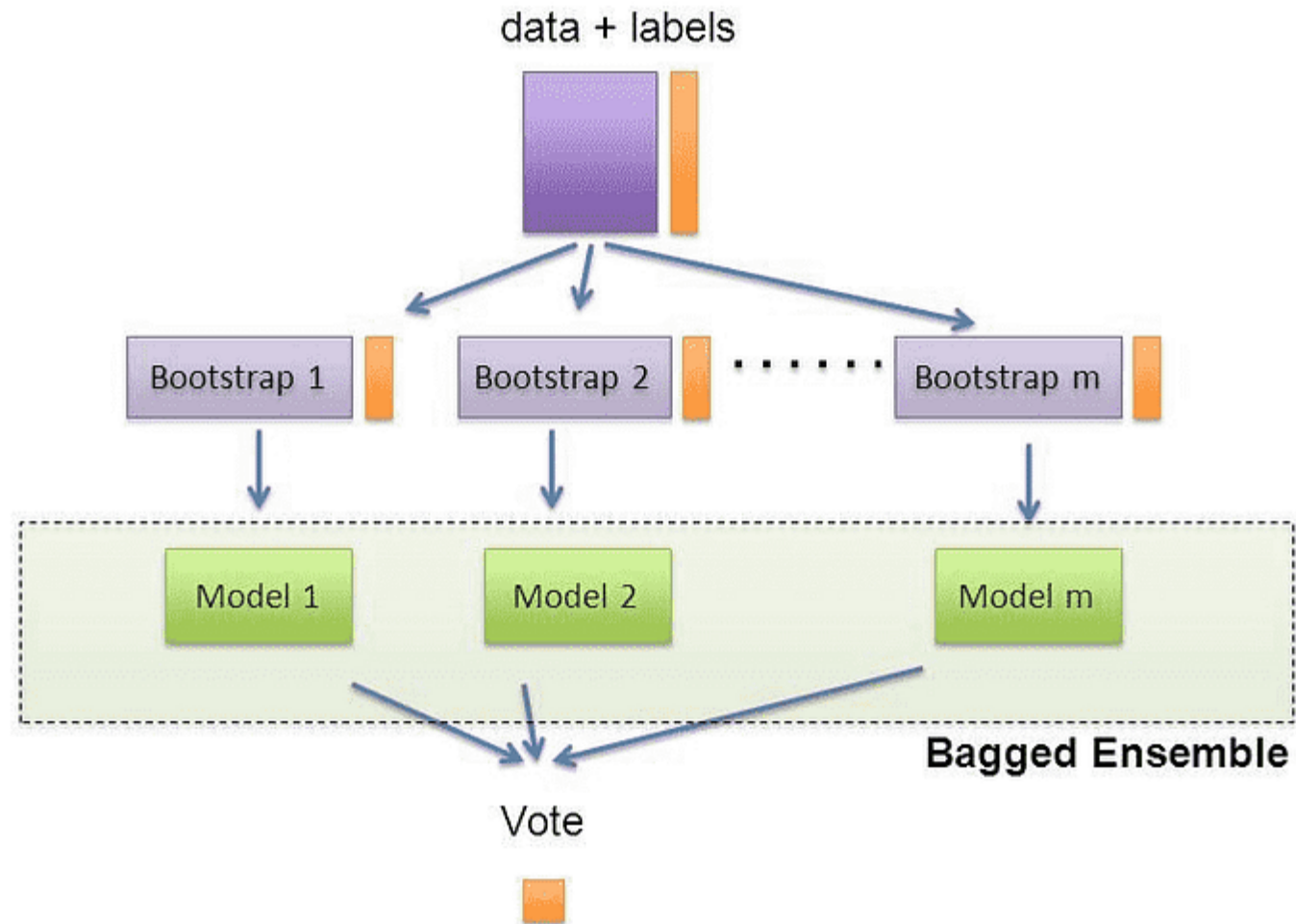
Ensemble Techniques

- **Bagging**

- Bagging is the type of ensemble technique in which a single training algorithm is used on different subsets of the training data where the subset sampling is done with replacement (bootstrap).
- Once the algorithm is trained on all the subsets, then bagging makes the prediction by aggregating all the predictions made by the algorithm on different subsets.
- In case of regression, bagging prediction is simply the mean of all the predictions and in the case of classifier, bagging prediction is the most frequent prediction (majority vote) among all the predictions.



“Bagging” : Bootstrap **AGG**regating

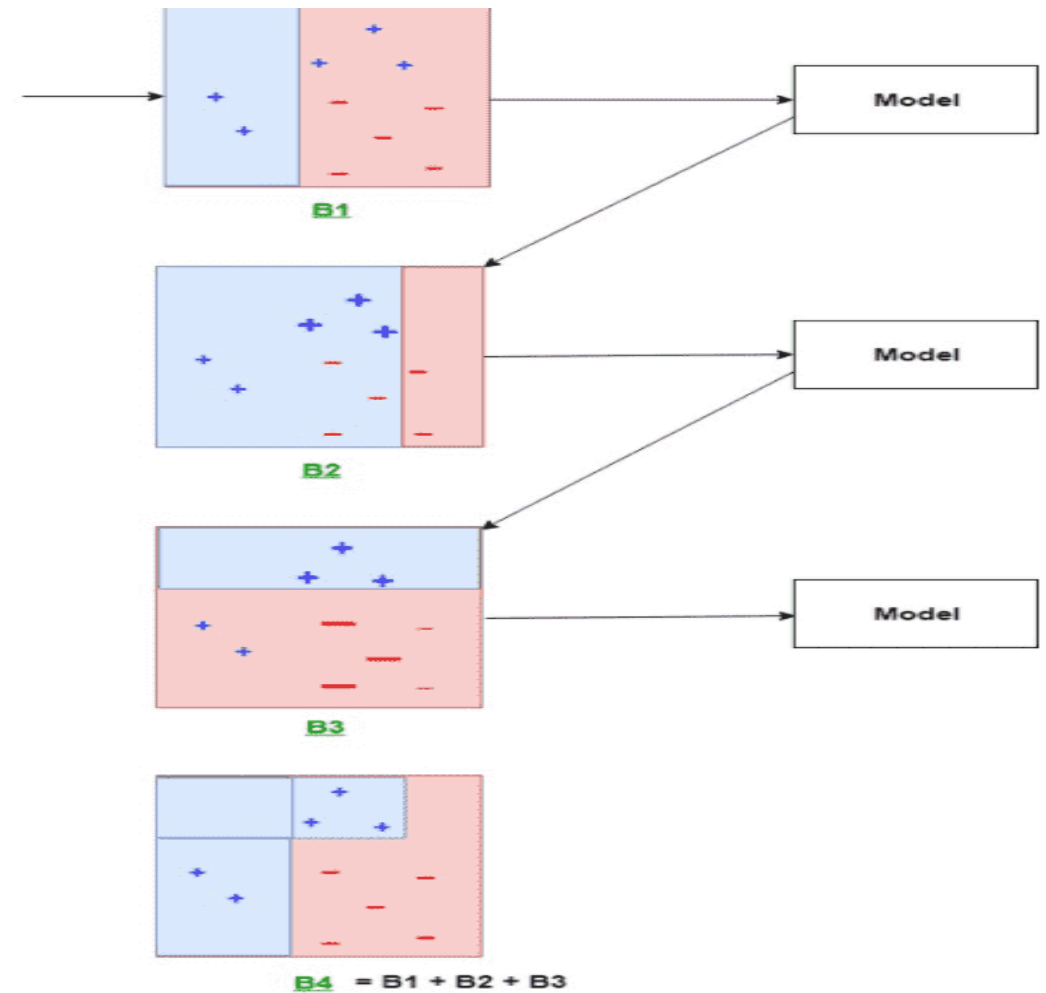


Boosting

- Boosting is an ensemble modeling technique which attempts to build a strong classifier from the number of weak classifiers.
- It is done building a model by using weak models in series. Firstly, a model is built from the training data.
- Then the second model is built which tries to correct the errors present in the first model. This procedure is continued and models are added until either the complete training data set is predicted correctly or the maximum number of models are added.



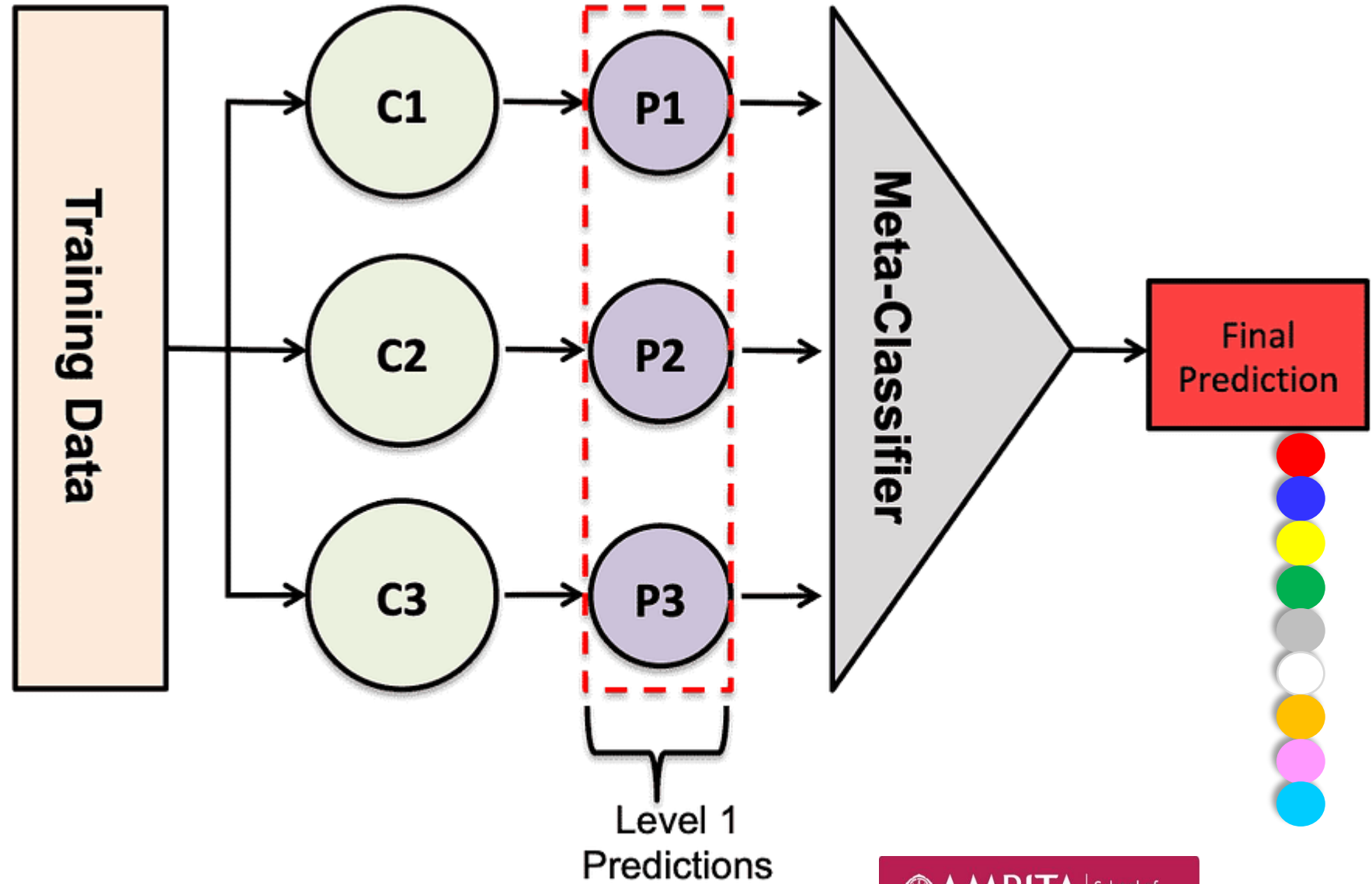
- B1 consist of **10 data points** which consist of two types namely plus(+) and minus(-) and **5 of which are plus(+)** and other **5 are minus(-)** and each one has been assigned **equal weight initially**. The first model tries to classify the data points and generates a vertical **separator line** but it **wrongly classifies 3 plus(+)** as minus(-).
- B2 consists of the 10 data points from the previous model in which the **3 wrongly classified plus(+)** are **weighted more** so that the current model tries more to classify these pluses(+) correctly. This model generates a vertical separator line which correctly classifies the previously wrongly classified pluses(+) **but in this attempt, it wrongly classifies three minuses(-)**.
- B3 consists of the 10 data points from the previous model in which the **3 wrongly classified minus(-)** are **weighted more** so that the current model tries more to classify these minuses(-) correctly. This model generates a horizontal separator line which correctly classifies the previously wrongly classified minuses(-).
- B4 combines together B1, B2 and B3 in order to build a strong prediction model which is much better than any individual model used. “usually 1 decision stumps”



Stacking

- Stacking is a type of ensemble technique which combines the predictions of two or more models, also called base models, and use the combination as the input for a new model (meta-model) i.e. a new model is trained on the predictions of the base models.
- Suppose you have a classification problem and you can use several models like logistic regression, SVM, KNN, Random forest etc. The idea is to use few models like KNN, SVM as the base model and make predictions using these models. Now the predictions made by these models are used as an input feature for Random forest to train on and give prediction.





* C1, C2, and C3 are considered level 1 classifiers.

Class Activity

- [Random Forest. Random Forest is a popular machine... | by Antony Christopher | Analytics Vidhya | Medium](#)
- Read the whole Blog for RF Classifier, Concentrate more on hyperparameter tuning.



Hyperparameters of Random forest classifier

- `RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None)`
- *Submit a single page write up on all these above parameters with explanation and explain below code snippet*
- **`from`** sklearn.ensemble **`import`** RandomForestClassifier
`rndmFrst = RandomForestClassifier(criterion='entropy',
max_depth = 12,
max_features = 'log2',
min_samples_leaf = 1,
min_samples_split= 5,
n_estimators = 90,random_state=6)`



CASE STUDY

A retail company wants to understand customer behavior in order to improve its marketing strategies. As a data analyst, you are required to perform a case study using supervised machine learning techniques for regression and classification. Download a simple customer dataset from Kaggle (such as a customer purchasing or spending dataset) and load it into Python for analysis. Perform exploratory data analysis to understand the dataset, preprocess the data by handling missing values and scaling features, and split the dataset into training and testing sets. Using the dataset, build regression models to predict a numerical value (such as customer spending or purchase amount) and classification models to categorize customers (such as high-value vs low-value customers). Implement at least two regression algorithms and two classification algorithms, evaluate their performance using suitable metrics, and compare the results. Finally, visualize the outcomes and briefly discuss how the machine learning models can help the company make better business decisions.

